SECURE CODING REVIEW REPORT

CYBER SECURITY – TASK 3

Submitted By:

Name: DHARANI V

Task Name: Secure Coding Review
Programming Language: JAVASCRIPT
Application Selected: Secure Coding Analysis

Date Of Submission: MAY 2026

OBJECTIVE:

The objective of this task is to perform a secure coding review on a web application to identify security vulnerabilities, analyze risks, and provide remediation techniques for safer software development. The selected application for this review is a Login Authentication Website developed using HTML, CSS, and JavaScript.

INTRODUCTION:

Secure coding is the practice of developing software that protects applications from security threats and vulnerabilities. A secure coding review helps identify weaknesses in source code before deployment.

In this task, a Login Authentication Website was created and reviewed to identify possible vulnerabilities in authentication and input handling mechanisms.

The application contains a login page where users enter a username and password for authentication.

SELECTED PROGRAMMING LANGUAGE AND APPLICATION:

Programming Language:

JavaScript

Technologies Used:

- HTML
- CSS
- JavaScript

Selected Application:

Login Authentication Website

Purpose of the Application:

The application allows users to log in using predefined credentials and displays a success or failure message based on authentication results.

APPLICATION DEVELOPMENT:

A simple Login Authentication Website was created using HTML, CSS, and JavaScript.

Features Implemented:

- User login interface
- Username and password authentication
- Show/Hide password option
- Validation messages
- Responsive user interface

Login Credentials Used:

Username: admin
Password: admin123

SECURE CODING REVIEW PROCESS:

The code review process was performed using:

1. Manual Code Inspection:

The source code was manually reviewed to identify security issues and insecure coding practices.

2. Static Analysis Method:

The JavaScript code was inspected for vulnerabilities using manual analysis and secure coding principles.

SECURITY VULNERABILITIES IDENTIFIED:

Vulnerability 1: Hardcoded Credentials

Description:

The username and password are directly stored inside the JavaScript code.

Vulnerable Code:

```
const correctUsername = "admin";

const correctPassword = "admin123";
```

Risk Level:

High

Impact:

An attacker can inspect the browser source code and view login credentials, leading to unauthorized access.

Recommendation:

Store credentials securely in a database and use encrypted password storage.

Vulnerability 2: Weak Authentication Mechanism

Description:

The authentication system is purely client-side and does not involve backend verification.

Risk Level:

High

Impact:

Attackers can bypass login authentication using browser developer tools.

Recommendation:

Implement server-side authentication using secure backend technologies.

Vulnerability 3: Weak Password Policy

Description:

The password used is simple and easy to guess.

Risk Level:

Medium

Impact:

Weak passwords increase the chances of brute force attacks.

Recommendation:

Use strong password policies such as:

- Minimum 8 characters
- Uppercase letters
- Numbers
- Special characters

Vulnerability 4: No Password Encryption

Description:

Passwords are stored in plain text.

Risk Level:

High

Impact:

Sensitive information may be exposed if source code is accessed.

Recommendation:

Use password hashing techniques such as bcrypt or SHA encryption.

Vulnerability 5: Lack of Backend Validation

Description:

All validation is performed only on the client side.

Risk Level:

Medium

Impact:

Users can manipulate code and bypass restrictions.

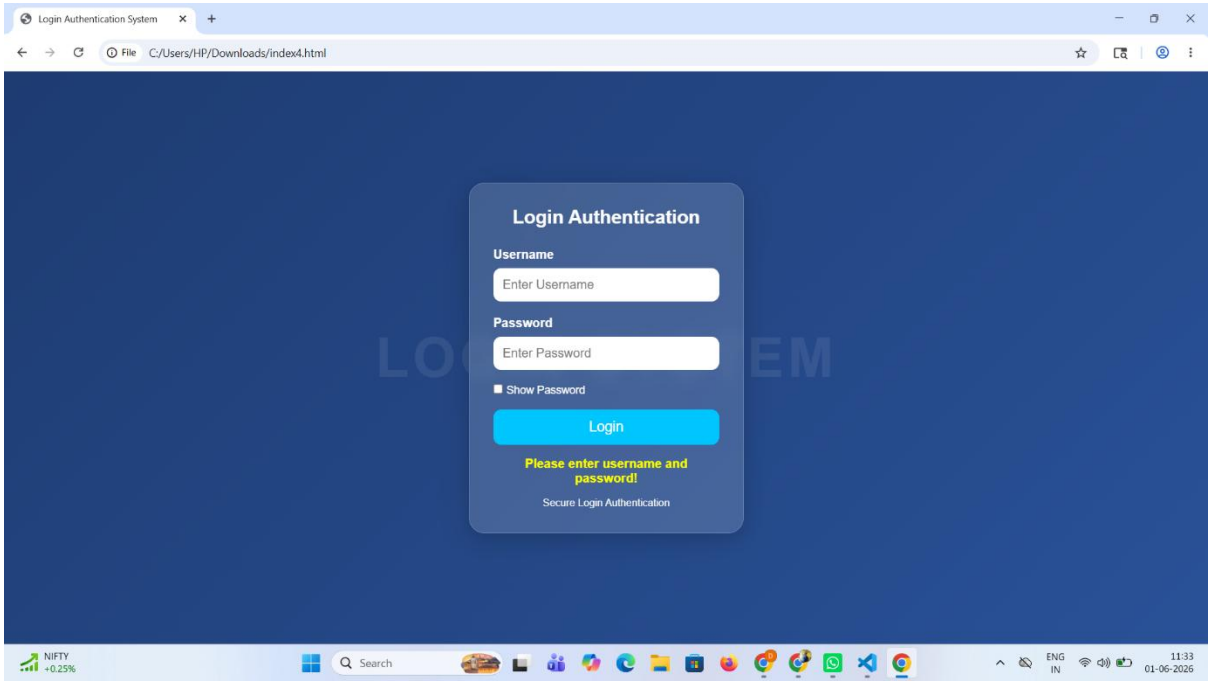
Recommendation:

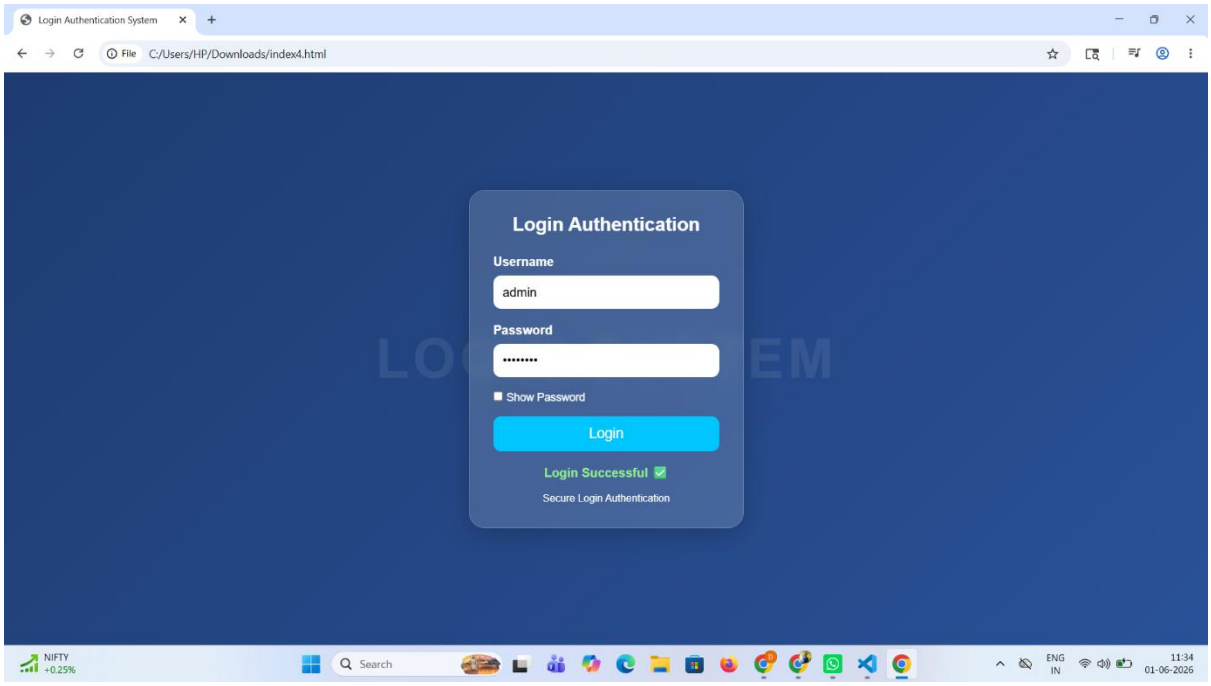
Perform server-side validation for authentication and form inputs.

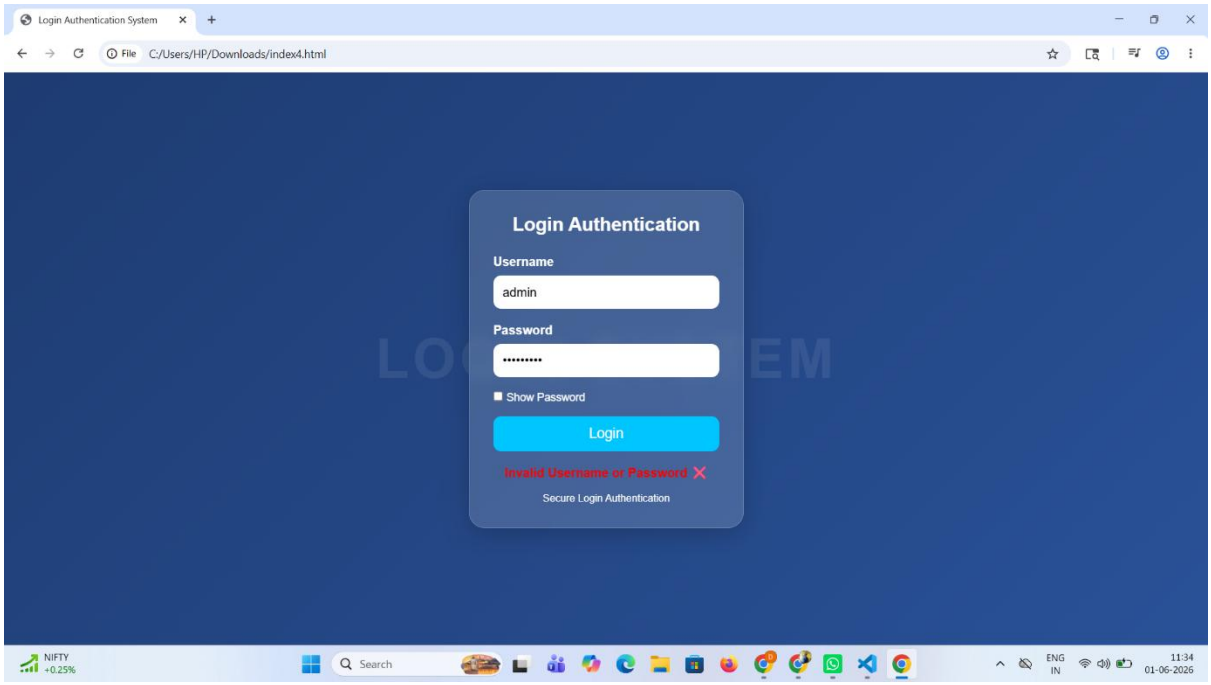
SECURITY ANALYSIS TABLE:

Vulnerability	Risk Level	Impact	Recommendation
Hardcoded Credentials	High	Unauthorized access	Use database authentication
Weak Authentication	High	Login bypass	Add backend authentication
Weak Password Policy	Medium	Easy password guessing	Use strong passwords
No Password Encryption	High	Password exposure	Encrypt passwords
No Backend Validation	Medium	Security bypass	Add server-side validation

SCREENSHOTS:







BEST PRACTICES FOR SECURE CODING:

The following secure coding practices are recommended:

1. Avoid hardcoded credentials in source code.
2. Validate all user inputs.
3. Use strong passwords.

4. Encrypt sensitive data.
5. Implement server-side authentication.
6. Use HTTPS for secure communication.
7. Apply access control mechanisms.
8. Regularly test applications for vulnerabilities.
9. Follow secure coding standards.
10. Keep software and dependencies updated.

REMEDIATION STEPS:

The following steps can improve application security:

- Replace hardcoded credentials with database storage.
- Use secure backend authentication.
- Encrypt passwords before storage.
- Add input validation and sanitization.
- Enable HTTPS communication.
- Implement session management.

RESULT:

The secure coding review successfully identified multiple vulnerabilities in the Login Authentication Website. Appropriate remediation steps and secure coding practices were suggested to improve the safety and reliability of the application.

CONCLUSION:

The Secure Coding Review task provided practical knowledge about identifying vulnerabilities in source code and implementing secure coding practices. Through manual

inspection of a Login Authentication Website, multiple security flaws such as hardcoded credentials, weak authentication, and lack of encryption were identified.

By applying secure coding standards and proper remediation methods, applications can be protected against cyber threats and unauthorized access.